DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 05

Lab Outcome: Implement backend services in Kotlin with database integration and secure authentication.

Aim: Write Kotlin programs to demonstrate data types, collections, and OOP concepts (classes, inheritance).

Software used: Kotlin

Theory: -

1. Introduction to Kotlin

Kotlin is a modern, statically typed programming language developed by JetBrains. It is widely used for Android application development, backend development, web applications, desktop applications, and multiplatform applications. Kotlin is designed to be concise, safe, modern, easy to learn, and fully interoperable with Java.

Simple analogy — a programming language is used to communicate with a computer. Just as English is used to communicate with people, Kotlin is used to communicate instructions to a computer. For example, println("Hello World") means: "Computer, display Hello World on the screen."

2. Software and Tools Required

Software	Purpose
JDK	Provides the Java environment required to compile/run Kotlin
Kotlin Compiler	Converts Kotlin source code into executable code
Visual Studio Code	Code editor used to write Kotlin programs

Software	Purpose
VS Code Kotlin Extension	Provides Kotlin-related editing support
Command Prompt / Terminal	Used to compile and execute programs

Recommended setup:

```
Kotlin Program
      ↓
Visual Studio Code
      ↓
Kotlin Compiler
      ↓
     JDK
      ↓
Program Execution
      ↓
    Output
```

3. Installing the JDK

JDK stands for Java Development Kit. Kotlin programs run on the Java Virtual Machine (JVM), so a suitable JDK is required. For beginners, use a current LTS JDK, such as JDK 21, downloaded from a trusted provider such as Eclipse Adoptium Temurin JDK or Oracle Java Downloads.

4. Installing JDK on Windows

1. Download the Windows installer corresponding to your computer architecture (Windows x64 for most modern computers).
2. Run the installer, keeping the default settings unless your institution has specified otherwise.
3. After installation, open Command Prompt (Windows + R, type cmd, press Enter).
4. Verify the installation:

```
java -version
```

Expected output (the exact version may differ):

```
java version "21.x.x"
```

Check the Java compiler:

```
javac -version
```

```
javac 21.x.x
```

If both commands work, the JDK installation is successful.

5. Installing Visual Studio Code

Visual Studio Code (VS Code) is recommended for this experiment because it is lightweight and beginner-friendly. Download it from the official Visual Studio Code website and install it using the normal installation options.

6. Install Kotlin Support in VS Code

1. Open VS Code and go to Extensions (or press Ctrl + Shift + X).
2. Search for "Kotlin".
3. Install a Kotlin language-support extension appropriate for your environment.

Important: Kotlin support in VS Code is not as comprehensive as the dedicated Kotlin/IntelliJ IDEA environment. For this introductory command-line experiment, however, VS Code is perfectly suitable for writing .kt files, while compilation and execution are performed through the Kotlin compiler in the terminal.

7. Installing the Kotlin Compiler

The Kotlin compiler converts Kotlin source code into a compiled program. Download Kotlin from the official Kotlin website and install the Kotlin command-line compiler for a simple lab setup. After installation, configure the Kotlin bin directory in the system PATH. The exact installation path depends on where Kotlin is extracted/installed, and may look conceptually like:

```
C:\Kotlin\kotlinc\bin
```

8. Configure the PATH

PATH is an operating-system setting that tells the computer where to look when a command is typed — for example, when kotlinc is typed, Windows needs to know where kotlinc is located.

Windows PATH configuration

1. Search Windows for "Environment Variables" and select Edit the system environment variables.
2. Go to Environment Variables → Path → Edit → New.
3. Add the bin directory of your Kotlin compiler, e.g. C:\Kotlin\kotlinc\bin.
4. Click OK → OK → OK, then close and reopen Command Prompt.

9. Verify Kotlin Installation

Open a new Command Prompt and type:

```
kotlinc -version
```

If Kotlin is correctly configured, this shows the Kotlin compiler version. Also test:

```
kotlin -version
```

If both commands are recognized, the environment is ready.

10. Create the First Kotlin Project

Create a folder named KotlinLab (for example, C:\KotlinLab). Open VS Code, select File → Open Folder, and choose KotlinLab.

11. Create the First Kotlin Program

Inside the folder, create Hello.kt and enter:

```
fun main() {
    println("Hello, Kotlin!")
}
```

Save the file.

12. Understanding the First Program

- fun — means we are defining a function. A function is a named block of instructions.
- main() — the starting point of the program. Think of it as: "Start executing the program from here."
- println("Hello, Kotlin!") — prints text on the screen.

13. Compile the Program

Open the VS Code terminal (Terminal → New Terminal, or Ctrl + `), navigate to the project folder if necessary, and compile:

```
kotlinc Hello.kt -include-runtime -d Hello.jar
```

This command means:

Part	Meaning
kotlinc	Kotlin compiler
Hello.kt	Input Kotlin file
-include-runtime	Include the required Kotlin runtime
-d Hello.jar	Create an output file named Hello.jar

14. Run the Program

```
java -jar Hello.jar
```

Expected output:

```
Hello, Kotlin!
```

The overall workflow is therefore:

```
Write code
   ↓
 .kt file
   ↓
 Compile
   ↓
 .jar file
   ↓
   Run
   ↓
 Output
```

Part A — Data Types

15. What is a Data Type?

A data type tells the computer what kind of data a variable stores. For example, 25 is a number, "Vijesh" is text, and true is a Boolean value.

16. Variables in Kotlin

Kotlin provides two important keywords: val and var.

val

A val variable cannot be reassigned after initialization:

```
val age = 20
```

Think: val = fixed reference.

var

A var variable can be changed:

```
var age = 20
age = 21
```

Think: var = variable/changeable.

17. Common Kotlin Data Types

Data Type	Purpose	Example
Int	Integer numbers	25

Data Type	Purpose	Example
Long	Large integers	100000L
Float	Decimal numbers	10.5f
Double	More precise decimal numbers	10.5
Char	Single character	'A'
String	Text	"Kotlin"
Boolean	True/false	true

18. Program 1 — Demonstrating Data Types

Create DataTypes.kt:

```
fun main() {

    val age: Int = 20
    val population: Long = 1500000L
    val percentage: Float = 85.5f
    val price: Double = 999.99
    val grade: Char = 'A'
    val name: String = "Vijesh"
    val passed: Boolean = true

    println("Name: $name")
    println("Age: $age")
    println("Population: $population")
    println("Percentage: $percentage")
    println("Price: $price")
    println("Grade: $grade")
    println("Passed: $passed")
}
```

19. Compile and Run

```
kotlinc DataTypes.kt -include-runtime -d DataTypes.jar
```

```
java -jar DataTypes.jar
```

Expected output:

```
Name: Vijesh
Age: 20
Population: 1500000
Percentage: 85.5
Price: 999.99
```

```
Grade: A
Passed: true
```

20. Important Concept — Type Inference

Kotlin can automatically identify the data type. Instead of:

```
val age: Int = 20
```

we can write:

```
val age = 20
```

Kotlin understands that 20 → Int. Similarly, val name = "Vijesh" is understood as "Vijesh" → String. This is called type inference.

Part B — Collections

21. What is a Collection?

Suppose we want to store Student 1, Student 2, Student 3, Student 4, and Student 5 — creating five separate variables is inconvenient. Instead, a collection allows us to store multiple values together. Kotlin provides several important collection types: List, Set, and Map.

22. List

A List stores elements in a particular order:

```
val students = listOf("Amit", "Neha", "Vijesh")
```

Conceptually:

```
Index       Value
  0         Amit
  1         Neha
  2         Vijesh
```

The index starts from 0.

23. Set

A Set stores unique elements:

```
val numbers = setOf(10, 20, 30, 10)
```

The duplicate 10 is not stored twice — conceptually the set holds 10, 20, 30.

24. Map

A Map stores data as Key → Value, for example Roll No → Student Name:

```
val students = mapOf(
    101 to "Amit",
    102 to "Neha",
    103 to "Vijesh"
)
```

Think of it like a dictionary: 101 → Amit, 102 → Neha, 103 → Vijesh.

25. Program 2 — Demonstrating Collections

Create CollectionsDemo.kt:

```
fun main() {

    // LIST
    val students = listOf(
        "Amit",
        "Neha",
        "Vijesh",
        "Priya"
    )

    println("Students:")
    for (student in students) {
        println(student)
    }

    // SET
    val numbers = setOf(10, 20, 30, 20, 10)

    println("\nUnique Numbers:")
    for (number in numbers) {
        println(number)
    }

    // MAP
    val studentMarks = mapOf(
        "Amit" to 85,
        "Neha" to 92,
        "Vijesh" to 78
    )

    println("\nStudent Marks:")
    for ((name, marks) in studentMarks) {
        println("$name : $marks")
    }
}
```

26. Compile and Execute

```
kotlinc CollectionsDemo.kt -include-runtime -d CollectionsDemo.jar
```

```
java -jar CollectionsDemo.jar
```

Expected output:

```
Students:
Amit
Neha
Vijesh
Priya

Unique Numbers:
10
20
30

Student Marks:
Amit : 85
Neha : 92
Vijesh : 78
```

27. Mutable Collections

There is an important distinction between listOf() and mutableListOf().

Immutable / read-only list

```
val students = listOf("Amit", "Neha")
```

We cannot add another element using add().

Mutable list

```
val students = mutableListOf("Amit", "Neha")

students.add("Vijesh")
```

The list now becomes: Amit, Neha, Vijesh.

Part C — Object-Oriented Programming

28. What is OOP?

OOP stands for Object-Oriented Programming — a programming approach where programs are organized around objects. Real-world examples include a Student, a Car, a Bank Account, an

Employee, and a Book. Each object can have:

- **Properties —** things the object has. A student has a name, a rollNo, and an age.
- **Behaviours —** things the object can do. A student can study(), attendClass(), and writeExam().

29. Class

A class is a blueprint or template for creating objects.

> *Real-world analogy — think about a house blueprint. The blueprint is not the actual house; it describes rooms, doors, windows, and the kitchen. Similarly, a Student class can describe a name, roll number, age, and study behaviour.*

30. Object

An object is an actual instance created from a class:

```
Class → Student

Objects:
Student 1 → Amit
Student 2 → Neha
Student 3 → Vijesh
```

31. Program 3 — Class and Objects

Create ClassDemo.kt:

```
class Student(
    val name: String,
    val rollNo: Int,
    val age: Int
) {

    fun displayDetails() {
        println("Name: $name")
        println("Roll No: $rollNo")
        println("Age: $age")
    }
}

fun main() {

    val student1 = Student("Amit", 101, 20)
    val student2 = Student("Neha", 102, 21)

    println("Student 1")
    student1.displayDetails()
```

```
    println()

    println("Student 2")
    student2.displayDetails()
}
```

32. Understanding the Program

class Student creates a class named Student. The class contains name, rollNo, and age — these are called properties — and it also contains displayDetails(), which is a method/function.

Creating an object:

```
val student1 = Student("Amit", 101, 20)
```

```
Student Class
      ↓
Student Object
      ↓
Amit, 101, 20
```

33. Compile and Run

```
kotlinc ClassDemo.kt -include-runtime -d ClassDemo.jar
```

```
java -jar ClassDemo.jar
```

Expected output:

```
Student 1
Name: Amit
Roll No: 101
Age: 20

Student 2
Name: Neha
Roll No: 102
Age: 21
```

Part D — Inheritance

34. What is Inheritance?

Inheritance allows one class to acquire properties and behaviours from another class.

```
Animal
   ↓
  Dog
```

A dog is an animal. The Animal class might contain eat() and sleep(); the Dog class can inherit these behaviours and additionally have bark().

35. Parent and Child Classes

The class being inherited from is called the Parent / Superclass. The class that inherits is called the Child / Subclass:

```
Animal
  │
  ├── Dog
  └── Cat
```

Here, Animal is the Parent, and Dog and Cat are Children.

36. Important Kotlin Rule

In Kotlin, classes are final by default. Therefore, if a class must be inherited, we use open class:

```
open class Animal
```

Now another class can inherit it.

37. Program 4 — Demonstrating Inheritance

Create InheritanceDemo.kt:

```
open class Animal {

    fun eat() {
        println("Animal is eating")
    }

    fun sleep() {
        println("Animal is sleeping")
    }
}

class Dog : Animal() {

    fun bark() {
        println("Dog is barking")
    }
}

fun main() {
```

```
    val dog = Dog()

    dog.eat()
    dog.sleep()
    dog.bark()
}
```

38. Understanding the Inheritance Syntax

class Dog : Animal() means Dog inherits from Animal. Therefore, the Dog object can use eat() and sleep(), because these methods belong to Animal, and it can also use bark(), because bark() belongs to Dog.

39. Compile and Run

```
kotlinc InheritanceDemo.kt -include-runtime -d InheritanceDemo.jar
```

```
java -jar InheritanceDemo.jar
```

Expected output:

```
Animal is eating
Animal is sleeping
Dog is barking
```

40. Inheritance with Method Overriding

Inheritance becomes more powerful when the child class changes the behaviour of a parent method. The parent might say "Animal makes a sound", while the dog can provide its own implementation: "Dog barks".

41. Program 5 — Inheritance and Overriding

Create OverrideDemo.kt:

```
open class Animal {

    open fun sound() {
        println("Animal makes a sound")
    }
}

class Dog : Animal() {

    override fun sound() {
        println("Dog barks")
    }
```

```
}

fun main() {

    val animal = Animal()
    val dog = Dog()

    animal.sound()
    dog.sound()
}
```

Output:

```
Animal makes a sound
Dog barks
```

42. Understanding open and override

A parent method marked open fun sound() means this method can be overridden by a child class. A child method marked override fun sound() means: "I am replacing the parent's implementation with my own implementation."

43. Complete Mini Program — Student Management

Students can combine all the concepts into one small program. Create StudentManagement.kt:

```
open class Person(
    val name: String,
    val age: Int
) {

    open fun displayDetails() {
        println("Name: $name")
        println("Age: $age")
    }
}

class Student(
    name: String,
    age: Int,
    val rollNo: Int
) : Person(name, age) {

    override fun displayDetails() {
        println("Student Name: $name")
        println("Age: $age")
        println("Roll No: $rollNo")
    }
}
```

```
fun main() {

    val students = mutableListOf(
        Student("Amit", 20, 101),
        Student("Neha", 21, 102),
        Student("Vijesh", 20, 103)
    )

    println("STUDENT DETAILS")
    println("----------------")

    for (student in students) {
        student.displayDetails()
        println()
    }
}
```

44. Compile and Execute

```
kotlinc StudentManagement.kt -include-runtime -d StudentManagement.jar
```

```
java -jar StudentManagement.jar
```

Expected output:

```
STUDENT DETAILS
----------------
Student Name: Amit
Age: 20
Roll No: 101

Student Name: Neha
Age: 21
Roll No: 102

Student Name: Vijesh
Age: 20
Roll No: 103
```

45. What Students Have Learned

```
Kotlin
  │
  ├── Variables
  │     ├── val
  │     └── var
  │
```

```
├── Data Types
│     ├── Int, Long, Float, Double
│     ├── Char, String
│     └── Boolean
│
├── Collections
│     ├── List
│     ├── Set
│     └── Map
│
└── OOP
      ├── Class, Object
      ├── Properties, Methods
      ├── Inheritance
      └── Method Overriding
```

46. Compilation and Execution — Quick Reference

For every .kt file:

1. Create the source file, e.g. Program.kt.
2. Compile: kotlinc Program.kt -include-runtime -d Program.jar
3. Execute: java -jar Program.jar

Example:

```
DataTypes.kt
     ↓
kotlinc DataTypes.kt -include-runtime -d DataTypes.jar
     ↓
DataTypes.jar
     ↓
java -jar DataTypes.jar
     ↓
  Output
```

47. Suggested Folder Structure

```
KotlinLab/
│
├── Hello.kt
├── DataTypes.kt
├── CollectionsDemo.kt
├── ClassDemo.kt
├── InheritanceDemo.kt
├── OverrideDemo.kt
└── StudentManagement.kt
```

Compiled .jar files can either be kept separately or deleted after testing.

48. Common Beginner Errors

Error 1 — 'kotlinc' is not recognized

Cause

The Kotlin compiler's bin directory is not in PATH.

Solution

Add the Kotlin compiler bin directory to the Windows PATH and reopen the terminal.

Error 2 — 'java' is not recognized

Cause

The JDK is not installed, or its bin directory is not available through PATH.

Solution

Check the installation with:

```
java -version
javac -version
```

Error 3 — File not found

If kotlinc DataTypes.kt ... cannot find the file, make sure the terminal is currently inside the folder containing DataTypes.kt. Check the folder contents using:

```
dir
```

Error 4 — Kotlin inheritance error

If you write:

```
class Animal
class Dog : Animal()
```

Kotlin will complain because classes are final by default. Use:

```
open class Animal
```

Implementation

Experiment Title: AIML Club Hackathon Management System

Problem Statement

Design and implement an object-oriented event registration module in Kotlin for an upcoming AIML Club Hackathon. The system must represent different participant profiles, calculate category-

specific registration fees using class inheritance and method overriding, and manage registrations using Kotlin Collections (List, Set, Map). It should demonstrate basic data types, immutability, polymorphism, and collection processing operations like filtering and aggregation.

File 1: Domain Models (Models.kt)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-5 > Models.kt

```kotlin
// Base class demonstrating encapsulation and inheritance preparation
open class Participant(
    val id: String,                    // Primitive String
    val name: String,
    val email: String,
    val experienceYears: Int,          // Primitive Int
    val skills: Set<String>            // Collection: Unique Set of skills
) {
    // Member property
    open val baseFee: Double = 1000.0 // Primitive Double

    // Method to be overridden by child classes (Polymorphism)
    open fun calculateFee(): Double {
        return baseFee
    }

    open fun getDetails(): String {
        return "[$id] $name | Category: General | Fee: ₹${calculateFee()}"
    }
}

// Subclass 1: Student Participant with fee discount
class StudentParticipant(
    id: String,
    name: String,
    email: String,
    experienceYears: Int,
    skills: Set<String>,
    val collegeName: String,
    val isACMClubMember: Boolean       // Primitive Boolean
) : Participant(id, name, email, experienceYears, skills) {

    override fun calculateFee(): Double {
        // 40% discount for students; extra 10% off for ACM members
        val discountRate = if (isACMClubMember) 0.50 else 0.40
        return baseFee * (1.0 - discountRate)
    }

    override fun getDetails(): String {
        return "[$id] $name ($collegeName) | Category: Student | Fee: ₹${calculateFee()}"
    }
}

// Subclass 2: Professional Participant with base fee surcharge
class ProfessionalParticipant(
    id: String,
    name: String,
    email: String,
    experienceYears: Int,
    skills: Set<String>,
    val companyName: String
) : Participant(id, name, email, experienceYears, skills) {
```

```
53
54      override fun calculateFee(): Double {
55          // Corporate track incurs an additional platform surcharge
56          val corporateSurcharge = 500.0
57          return baseFee + corporateSurcharge
58      }
59
60      override fun getDetails(): String {
61          return "[$id] $name ($companyName) | Category: Professional | Fee: ₹${calculateFee()}"
62      }
63  }
```

File 2: System Controller (Main.kt)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-5 > Main.kt

```
1   fun main() {
2       println("=== AIML CLUB HACKATHON REGISTRATION SYSTEM ===\n")
3
4       // Instantiating objects using constructor inheritance
5       val p1 = StudentParticipant(
6           id = "STU-101",
7           name = "Aarav Sharma",
8           email = "aarav@tech.edu",
9           experienceYears = 2,
10          skills = setOf("Python", "Kotlin", "PyTorch"),
11          collegeName = "National Institute of Tech",
12          isACMClubMember = true
13      )
14
15      val p2 = StudentParticipant(
16          id = "STU-102",
17          name = "Ananya Iyer",
18          email = "ananya@univ.edu",
19          experienceYears = 1,
20          skills = setOf("Java", "Kotlin", "UI/UX"),
21          collegeName = "State Engineering College",
22          isACMClubMember = false
23      )
24
25      val p3 = ProfessionalParticipant(
26          id = "PRO-201",
27          name = "Rohan Verma",
28          email = "rohan@aimllabs.com",
29          experienceYears = 5,
30          skills = setOf("Kotlin", "TensorFlow", "Docker"),
31          companyName = "AI Innovation Labs"
32      )
33
34      // Collection 1: Polymorphic List storing base and subclass types
35      val registryList: List<Participant> = listOf(p1, p2, p3)
36
37      // Collection 2: Map for O(1) quick lookups using Participant ID key
38      val registryMap: Map<String, Participant> = registryList.associateBy { it.id }
39
40      // Displaying all registrations (Polymorphism in action)
41      println("--- Registered Participants ---")
42      registryList.forEach { participant ->
43          println(participant.getDetails())
44      }
45
46      // Collection Operation 1: Filtering participants with Kotlin skill
47      println("\n--- Kotlin Skilled Applicants ---")
48      val kotlinDevs = registryList.filter { "Kotlin" in it.skills }
49      kotlinDevs.forEach { println("• ${it.name} (${it.email})") }
```

```
50
51      // Collection Operation 2: Unique skills set aggregation across all participants
52      val allUniqueSkills: Set<String> = registryList.flatMap { it.skills }.toSet()
53      println("\n--- Skill Set Diversity ---")
54      println("Total Unique Skills: ${allUniqueSkills.size}")
55      println("Skill Index: ${allUniqueSkills.joinToString(", ")}")
56
57      // Aggregation: Calculate total revenue using sumOf
58      val totalRevenue = registryList.sumOf { it.calculateFee() }
59      println("\nTotal Projected Revenue: ₹$totalRevenue")
60
61      // Map Lookup Demonstration
62      val searchId = "STU-101"
63      println("\n--- Direct Search Lookup ($searchId) ---")
64      val result = registryMap[searchId]
65      if (result != null) {
66          println("Match Found: ${result.name} | Direct Contact: ${result.email}")
67      } else {
68          println("Participant ID not registered.")
69      }
70  }
```

How the Files Work Together

Models.kt defines the structural blueprints and business rules for hackathon participants, while Main.kt acts as the execution engine that creates, organizes, and processes participant data.

What Models.kt Does (Data Structure & Business Logic)

- Base Class (Participant): defines common attributes (ID, name, skills) and a default fee calculation method.
- Subclasses (StudentParticipant & ProfessionalParticipant): extend the base class using OOP inheritance, adding category-specific properties (college/company) and overriding calculateFee() to apply dynamic discounts or corporate surcharges.
- Separation of Concerns: contains zero execution code or console output statements — it purely models what a participant is and how their fee is calculated.

What Main.kt Does (Execution & Operations)

- Data Instantiation: creates concrete objects (p1, p2, p3) using the blueprints defined in Models.kt.
- Collection Storage: groups these objects into a List<Participant> for iteration and a Map<String, Participant> for key-value searching.
- Data Processing: performs functional queries like filtering participants by skill, compiling a unique list of all skills, and computing total revenue.

How the Two Files Connect

In Kotlin, all files residing in the same package share a common namespace. Because both files share the same default package, Main.kt can access classes like StudentParticipant from Models.kt directly, without writing import statements. During compilation, the Kotlin compiler (kotlinc) combines both source files into a unified executable.

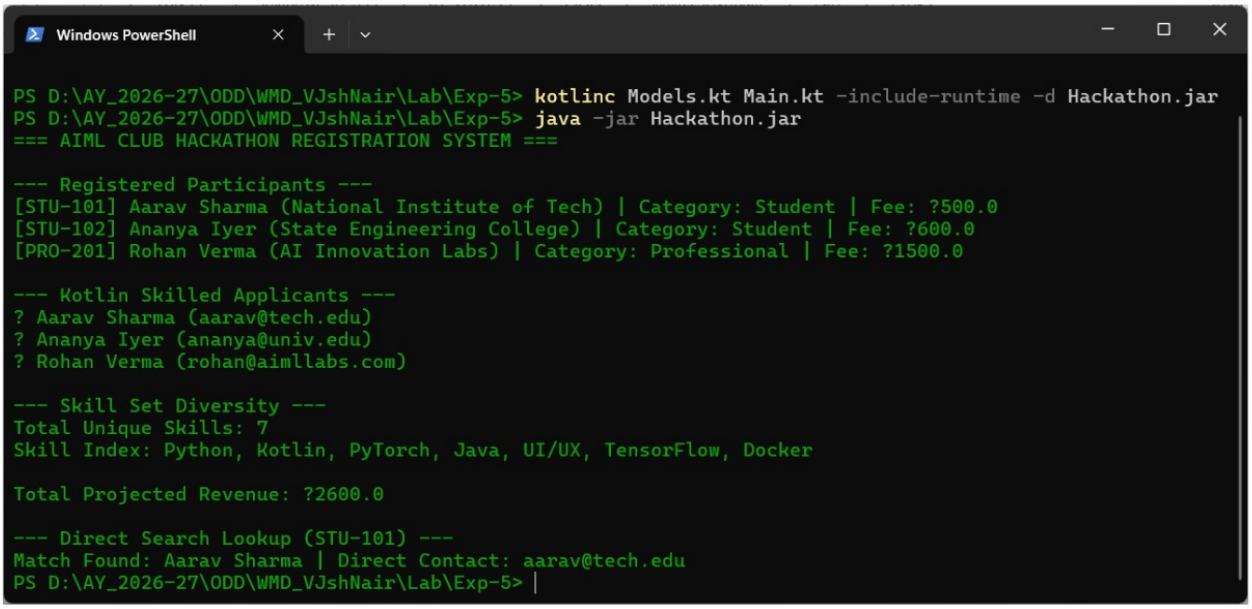
Data Flow Architecture

Step	Initiator	Target File	Action & Data Transferred
1. Instantiation	Main.kt	Models.kt	Main.kt passes raw argument values ("Aarav", true) into subclass constructors.
2. Object Creation	Models.kt	System Memory	Constructs instances in memory with calculated attributes based on overridden logic.
3. Aggregation	Main.kt	Main.kt	Wraps object instances into polymorphic collections (List<Participant>).
4. Logic Execution	Main.kt	Models.kt	Main.kt triggers .calculateFee() on each item; Kotlin invokes the child class logic at runtime.
5. Results Output	Models.kt	Main.kt / Console	Formatted strings and calculated values return to Main.kt to display terminal output.

Execution Flow, Step by Step

1. Program Launch: the JVM starts execution at fun main() inside Main.kt.
2. Object Memory Allocation: Main.kt invokes constructors from Models.kt. For p1, isACMClubMember = true triggers a 50% discount in StudentParticipant.
3. Polymorphic Collection Building: listOf(p1, p2, p3) stores all objects as Participant references, hiding class differences behind a single list interface.
4. Polymorphic Method Call: calling participant.getDetails() inside forEach dynamically invokes the specific .calculateFee() method for each participant type.
5. Collection Processing: .filter { "Kotlin" in it.skills } inspects each object's skill Set property and generates a filtered subset.
6. Key Lookup: registryMap["STU-101"] queries the Map using the participant ID, returning direct access to p1 to fetch contact details.

Expected Output



Lab Exercises for Students

After completing the demonstration, students should perform the following exercises independently.

Exercise 1 — Data Types

Write a Kotlin program to store and display: student name, roll number, age, CGPA, grade, and pass/fail status. Use appropriate Kotlin data types.

Exercise 2 — Collections

Create a list containing the names of five students. Perform:

1. Display all students.
2. Display the first student.
3. Display the last student.
4. Add a student using a mutable list.
5. Remove a student.
1. Display the final list.

Exercise 3 — Set

Create a set containing: 10, 20, 30, 20, 40, 10. Display the resulting set and observe what happens to duplicate values.

Exercise 4 — Map

Create a map containing Roll No → Student Name. Store at least five students and display all key-value pairs.

Exercise 5 — Class and Object

Create a class Book with title, author, and price, and a method displayDetails(). Create at least two objects and display their details.

Exercise 6 — Inheritance

Create Employee → Manager. The Employee class should contain name and salary. The Manager class should additionally contain department. Display all details.

Exercise 7 — Method Overriding

Create Shape → Circle and Shape → Rectangle. Create a method area() in the parent class and override it appropriately in each child class.

Conclusion: -

Thus, we have demonstrated and implemented data types, collections, and OOP concepts (classes, inheritance).